

Experiment: Map Coloring Problem using Constraint Satisfaction Problem (CSP)

Aim

To implement the **Map Coloring Problem** using the **Constraint Satisfaction Problem (CSP)** approach in Python by assigning different colors to adjacent regions.

Objective

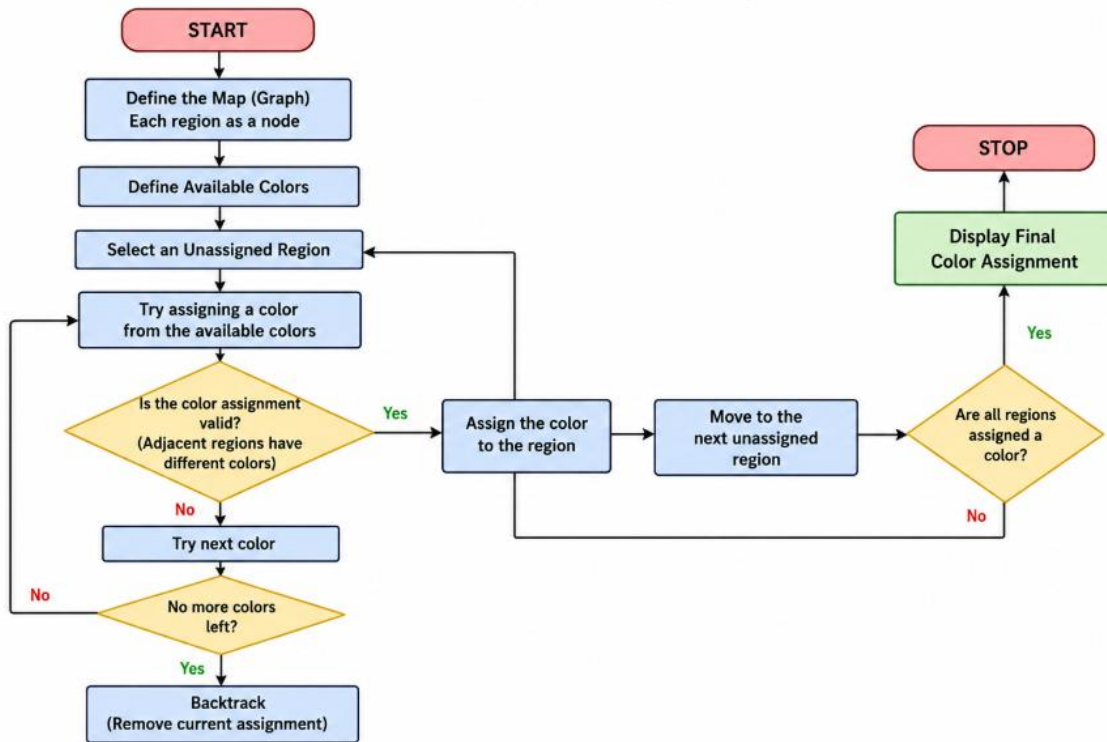
To assign colors to each region of a map such that no two adjacent regions have the same color while using the minimum number of colors.

Algorithm

1. Start.
2. Define the map as a graph where each node represents a region.
3. Define the available colors.
4. Select an unassigned region.
5. Assign a color that satisfies the constraints.
6. Check whether adjacent regions have different colors.
7. If the assignment is valid, continue with the next region.
8. If no valid color is available, backtrack.
9. Repeat until all regions are assigned colors.
10. Display the final color assignment.
11. Stop.

FLOWCHART

Flowchart: Map Coloring using CSP



CODE:

Map Coloring using Constraint Satisfaction Problem (CSP)

Define the map (graph)

```
graph = {  
    'A': ['B', 'C', 'D'],  
    'B': ['A', 'C', 'E'],  
    'C': ['A', 'B', 'D', 'E'],  
    'D': ['A', 'C', 'E'],  
    'E': ['B', 'C', 'D']  
}
```

Available colors

```

colors = ['Red', 'Green', 'Blue']

# Dictionary to store assigned colors
assignment = {}

# Check if assigning a color is valid
def is_valid(region, color):
    for neighbor in graph[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True

# Backtracking function
def map_coloring(region_list, index):
    if index == len(region_list):
        return True

    region = region_list[index]

    for color in colors:
        print(f"Trying {region} -> {color}")

        if is_valid(region, color):
            assignment[region] = color
            print(f"Assigned {region} = {color}")

            if map_coloring(region_list, index + 1):
                return True

```

```
        print(f"Backtracking from {region}")
        del assignment[region]

    return False

regions = list(graph.keys())

if map_coloring(regions, 0):
    print("\nFinal Map Coloring Solution")
    print("-----")
    for region in regions:
        print(region, ":", assignment[region])
else:
    print("No solution exists.")
```

OUTPUT:

```
Trying A -> Red
Assigned A = Red
Trying B -> Red
Trying B -> Green
Assigned B = Green
Trying C -> Red
Trying C -> Green
Trying C -> Blue
Assigned C = Blue
Trying D -> Red
Trying D -> Green
Assigned D = Green
Trying E -> Red
Assigned E = Red

Final Map Coloring Solution
-----
A : Red
B : Green
C : Blue
D : Green
E : Red
```

RESULT:

The Map Coloring problem was successfully solved using the **Constraint Satisfaction Problem (CSP)** approach with the **Backtracking algorithm**, ensuring that no two adjacent regions were assigned the same color.

Conclusion

The Python program successfully implemented the **Map Coloring Problem** using CSP and backtracking. The algorithm efficiently assigned colors to all regions while satisfying the adjacency constraints, demonstrating the effectiveness of backtracking in solving constraint satisfaction problems.